Generating Runtime-Verification Specs from Natural Language

Daniel Feng

April 29, 2026

1 Abstract

Runtime Verification (RV) is a dynamic software analysis process crucial for ensuring application reliability by continuously monitoring execution traces against formal logic requirements. However, translating human-readable natural language software rules into complex formal logic is notoriously difficult and time-consuming, creating a steep barrier to entry for the widespread adoption of RV tools. To address this bottleneck, this research presents an automated translation pipeline leveraging Large Language Models to convert natural language requirements into Extended Regular Expressions (ERE) specifically for the Monitoring-Oriented Programming (MOP) framework. The proposed architecture utilizes a dual-transformer preprocessing system—comprising an NL Clarifier and an Event Interpreter—to normalize ambiguous human documentation and raw code signatures into structured English, thereby reducing the cognitive load on the final logic generator. Additionally, to overcome the limitations of strict syntactic string-matching, this study introduces an automated Semantic Comparison Engine that mathematically proves the equivalence of generated and ground-truth logics by converting them into minimal deterministic finite automata (min-DFA). Evaluation against a dataset of annotated Java API properties demonstrated that the modular clarification pipeline significantly improved the semantic accuracy of the model, increasing it from 42% to 61%. Ultimately, this automated approach effectively lowers the barrier to entry for utilizing formal software specifications, paving the way for more accessible and rigorous real-time software debugging.

2 Introduction

2.1 Context & Motivation

Modern software engineering requires rigorous monitoring to ensure that applications function safely and reliably. One powerful method developers use to achieve this is Runtime Verification (RV). RV is the process of continuously monitoring a software application's runtime execution trace—the sequential log of events that occur as a program runs—and verifying that this sequence satisfies certain behavioral properties. Software engineers rely on RV to verify program requirements and catch logical violations in real-time. In a collaborative human environment, these rules are typically conceptualized and communicated in natural language. For instance, a developer might state a requirement as simply as "Every file opened must be closed" or "Never modify a collection when it is being accessed".

2.2 Problem Statement

The root cause of friction in the Runtime Verification pipeline is a fundamental communication barrier: computers cannot accept natural language. For an RV tool to track a program, these simple English requirements must be translated into "formal logic"—rigorous mathematical equations and code that explicitly define the boundaries of what is and is not allowed.

Writing formal language is notoriously difficult. It requires a deep understanding of complex state machines and mathematical syntax, making it highly error-prone and time-consuming for developers. Because formal logic is so hard to write, it presents a steep barrier to entry, limiting the widespread adoption of robust runtime verification tools in the software industry.

2.3 Objective & Scope

To address this bottleneck, this project sets out to automate the translation of natural language software requirements into formal logic by utilizing advanced transformer models (Large Language Models). Our research focuses specifically on the Monitoring-Oriented Programming (MOP) framework, an RV system that captures execution traces and generates dedicated monitors to analyze them. MOP relies on .mop files, which are structured into four main components: Parameterization, Events, Logic, and Handlers. While there are many different languages of formal logic used in computer science, the scope of this project is explicitly targeted at generating the Logic component using Extended Regular Expressions (ERE). To accomplish this, we utilize the OpenAI 5.5 API to power an automated translation pipeline. The system is evaluated by analyzing natural language properties sourced directly from annotated Java API (Javadoc) documentation.

3 Background

3.1 Runtime Verification and Monitoring-Oriented Programming (MOP)

Runtime Verification (RV) is a dynamic software analysis process used by engineers to observe a program as it runs and verify that its execution trace—the sequential sequence of events—satisfies specific behavioral requirements. Instead of just testing code before it is deployed, RV continuously evaluates real-time actions, such as ensuring a file is never modified while it is being accessed, to rigorously detect logical violations during actual software operation. Monitoring-Oriented Programming (MOP) is an RV framework that facilitates this process by capturing execution traces and generating specialized tools called "monitors" to evaluate them. Governed by structured specifications known as .mop files, these autonomous monitors track the execution trace as it grows, checking the sequence of events against predefined formal logic to confirm the program is behaving correctly. A MOP file is characterized by 4 parts: the parameterization, the events, the logic, and the handler. In this project, we focus on the logic.

3.2 Formal Specification Logics

Formal specification logics are rigorous, mathematically based languages used to unambiguously define the expected behaviors, temporal properties, and constraints of a software system. They are what a MOP file means by the "logic." Given a logic, automated verification tools like MOP can accurately and continuously monitor execution traces for logical violations.

3.2.1 Extended Regular Expressions (ERE)

Extended Regular Expressions (ERE) adapt standard pattern-matching mechanics to evaluate sequential software events rather than simple text characters. In addition to standard operators like Kleene closure ($*$), positive closure ($+$), and alternation ($|$), EREs in the context of runtime verification uniquely incorporate the empty event (ϵ) and account for non-terminating, theoretically infinite execution traces. A specific extension beyond RE is the existence of creation events and the deletion of terminating events.

3.2.2 Linear Temporal Logic (LTL)

Linear Temporal Logic (LTL) evaluates the state of a system over continuous, infinite paths of time using specific temporal modalities. It combines standard propositional logic operators like AND ($\wedge$) and implication ($\rightarrow$) with time-specific operators—such as "Globally" (G), "Next" (X), and "Until" (U)—to construct complex, time-dependent rules about future system states.

3.3 Evaluation of Logic

Evaluating formal logic is the process of determining whether two mathematical expressions, which may look entirely different on the surface, represent the exact same set of rules. To accurately compare the logic generated by our Large Language Models against the expected ground truths, we must establish a framework for equivalence.

3.3.1 Syntactic Similarity

Syntactic similarity measures the strict, character-by-character equivalence of two expressions. For instance, while the expression "ABB+" is syntactically equivalent to "ABB+" , it is syntactically dissimilar to the expression "ABB*", even though they are closely related.

3.3.2 Semantic Similarity

Semantic similarity measures whether two different expressions accept the exact same language and evaluate traces identically. In our previous example, the expressions "ABB*" and "AB+" are syntactically different, but they are semantically similar because both logics require the trace to consist of one "A" followed by one or more "B"s.

3.4 Related Work & Novelty

Existing research has explored using Large Language Models (LLMs) to translate natural language into formal logic, though primarily focusing on Linear Temporal Logic (LTL). For example, the nl2spec framework uses the Codex model to translate requirements into LTL, achieving a 58% baseline accuracy without human intervention. However, nl2spec relies on models fine-tuned on general GitHub repositories rather than specific runtime-verification logic, and it heavily requires manual expert validation due to the lack of an automated comparison tool. A more recent framework, Req2LTL, improves translation accuracy by decomposing natural language into an intermediate structured representation, called OnionL, before translating it to LTL. While these frameworks demonstrate the viability of transformer models in this domain, they do not address the specific structural requirements and behaviors of Extended Regular Expressions (ERE) used in Monitoring-Oriented Programming (MOP).

Because there is no prior automated framework designed explicitly to generate formal ERE languages for Runtime Verification, robust datasets pairing natural language with ERE specifications are scarce. To properly evaluate our proposed pipeline against a reliable ground truth, we utilize a manually curated dataset developed by Legunsun. This database contains 137 annotated natural language properties sourced directly from Java API documentation, paired alongside their corresponding formal MOP specifications. Additionally, during the initial exploration of model fine-tuning, datasets containing over 55,000

standard regular expression composition tasks from Çakar et al. were utilized in an attempt to adapt local models for this specific task.

This project builds upon these foundations by introducing a specialized pipeline that targets the unique nuances of ERE generation. Unlike previous methods that rely heavily on human experts to verify syntactic output strings, our approach introduces a rigorous, automated semantic comparison engine. By converting both the LLM-generated logic and the ground-truth logic into minimal deterministic finite automata (min-DFA), we can efficiently and mathematically prove semantic equivalence. This innovation not only allows for rapid testing over larger datasets without manual intervention, but also directly addresses the limitations of syntactic comparison found in prior translation tools.

4 Methods

4.1 Stakeholders and Success Criteria

The primary stakeholders for this project are software engineers and quality assurance testers who rely on Runtime Verification to ensure program reliability but struggle with the steep learning curve of writing formal logic. The core success criterion is the development of a functional, LLM-driven translation pipeline capable of autonomously generating accurate Extended Regular Expressions (ERE) from natural language software requirements. Furthermore, success requires the implementation of a robust semantic comparison engine that can mathematically verify the equivalence of these generated EREs against ground-truth datasets using minimal deterministic finite automata (min-DFA). Ultimately, achieving these goals will successfully lower the barrier to entry for utilizing Monitoring-Oriented Programming (MOP) in complex software environments.

4.2 Semantic Comparison Engine

To accurately evaluate the performance of the Large Language Model, it is insufficient to simply compare the generated logic against the ground truth character-by-character (syntactic similarity). Two structurally different expressions can enforce the exact same rules. Therefore, we engineered an automated Python evaluation pipeline to verify semantic equivalence by converting both Extended Regular Expressions (EREs) into minimal deterministic finite automata (min-DFA). Because every regular language has exactly one unique, canonical min-DFA representation, we can definitively prove semantic equivalence if the generated min-DFA is mathematically isomorphic to the ground-truth min-DFA. The engine operates through a sequential, multi-step algorithmic pipeline:

4.2.1 Event Anonymization

Standard finite state machine libraries are typically designed to process single-character alphabets rather than verbose event names like use_file or modify_file.

The engine first extracts all unique events from both the generated and ground-truth EREs and maps them to a shared dictionary of single characters (e.g., A, B, C). Epsilon events (ϵ), which indicate an empty trace, are also normalized. This ensures the expressions are uniformly formatted for algorithmic evaluation.

4.2.2 Creation Events

Execution traces theoretically grow indefinitely, but a MOP monitor only begins live-tracking when an explicitly defined "creation" event occurs. To model this, the engine injects a novel gateway state at the start of the min-DFA. This state serves as a mandatory prerequisite, connecting to the initial events via the creation event's transition. This ensures expressions like A^* are correctly processed as requiring at least one A ($A+$).

4.2.3 Terminating Events

If a MOP logic expression triggers a matching handler (@match), the monitor activates the moment the pattern is fulfilled; any subsequent events in the trace are irrelevant. To account for this, the engine iterates through all accepting (terminal) states in the min-DFA and programmatically deletes their outbound transition edges. Following this truncation, a root-node graph search is executed to identify and permanently delete any states or transitions that have been rendered unreachable.

4.2.4 Equivalence Verification

If a MOP logic expression triggers a matching handler (@match), the monitor activates the moment the pattern is fulfilled; any subsequent events in the trace are irrelevant. To account for this, the engine iterates through all accepting (terminal) states in the min-DFA and programmatically deletes their outbound transition edges. Following this truncation, a root-node graph search is executed to identify and permanently delete any states or transitions that have been rendered unreachable.

4.3 Iterative Prompt Engineering & Fine-Tuning

Because standard Large Language Models are not natively trained to write Monitoring-Oriented Programming (MOP) specifications, we utilized rigorous prompt engineering to guide the model's output. The generation process was structured as an iterative feedback loop: we supplied an initial prompt to the OpenAI API, evaluated the generated logic against a small preliminary test set, analyzed the remaining issues, and revised the prompt instructions accordingly.

This methodology integrated general principles from OpenAI's official prompt engineering guidelines and extended insights from B. Çakar's studies on standard regular expression generation into the Extended Regular Expression (ERE) domain.

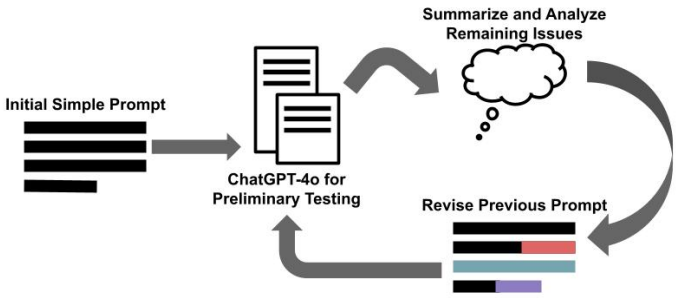

Figure 1: Flowchart showing how the prompt was iteratively refined

Through six major iterations (Iteration 0 to Iteration 5), the prompt evolved from a simple natural language translation request into a highly specific, constraint-bound system prompt. Iterative refinements are detailed in the table:

Table 1: Iterative Prompt Engineering Refinements and Performance

Modifications	**Iteration**	**Prompt Addition/Change**
	Iteration #0 (Accuracy 0/3)	Translate the following natural language into regular expression for java runtime-verification

Table 1 – continued from previous page

Modifications	Iteration	Prompt Addition/Change
Defines @fail and @match handlers. Provides ERE syntax dictionary. Elaborates more on characteristics of MOP (only events defined will be considered in the trace).	Iteration #1 (Accuracy 0/2)	The result of the regular expression will fall into two categories: match, meaning the expression matches the entire relevant trace, or fail, meaning the expression does not match onto the entire relevant trace. It should be indicated for the final expression which one of these results maps to a violation of the runtime property. Recall that regular expression uses "(" and ")" to group events, "|" to signify "or", "*" to signify match as many as possible, and "+" to signify match as many as possible but at least once. For example, the expression "(A|B)+" means match either event A or event B as many times as possible but at least once. In the regular expression, do not pass arguments to the events and only put the event name. Further, the expression will ignore any event not defined, so using ".*" is unnecessary.
Further elaborated on "only events defined will be considered in the trace." Elaborated on the logic that comes with "creation" events.	Iteration #2 (Accuracy 1/3)	Any events not named in the regular expression are automatically ignored and need not be accounted for using .*, other*, or any wildcard inside of the expression. The created regular expression should be as specific as possible and should begin from the creation of the object(s) in question.
Discourage usage of the ? operator and instead using epsilon.	Iteration #3 (Accuracy 1/3)	On the otherhand, operators like "?" and "{a,b}" do not exist and should not be used. For example, the expression "(A|B)+" means match either event A or event B as many times as possible but at least once. Furthermore, this special form of Regex has added an event called "epsilon", which represents the event of nothing. For example, the expression "(A|epsilon)" would match both the trace "A" and an empty trace.

Table 1 – continued from previous page

Modifications	Iteration	Prompt Addition/Change
Explicitly stated that all interactions should be considered in the ERE (A before B, B before A).	Iteration #4 (Accuracy 2/3)	To be able to match the relevant trace, analyze how the events in the natural language interact with each other at all times from creation. For example, even if the natural language specifies that event A cannot occur after event B, make sure to consider event A occurring before event B in order to consider the entire trace.
Explicitly stated that the trace can be unconventional, for example "shutdown" can be in the trace multiple times.	Iteration #5 (Accuracy 1/3)	In the trace, unconventional sequences events may occur, for example events like "terminate" and "start" could be ran multiple times. If the following natural language does not specify against it, cases like these need to considered.

4.4 Natural Language and Code Clarification

A fundamental challenge in automating specification generation is bridging the semantic gap between human-written documentation and machine-readable code. Large Language Models (LLMs) perform optimally when given explicit, highly structured context; however, real-world inputs rarely fit this mold. To optimize the input data before it reaches the final logic generator, our architecture employs two independent preprocessing transformers: the NL Clarifier and the Event Interpreter.

In real-world software documentation, such as Java API Javadocs, natural language requirements are often terse, vague, and highly contextual. For example, a dataset property might simply state:

> Prevents invocations of mark() if the Reader does not support.

Compared to the highly structured, mathematically precise prompts utilized in other related temporal logic frameworks, this ambiguous phrasing forces the generative model to guess the missing operational parameters.

To address this, the pipeline utilizes the NL Clarifier model to preprocess and expand the raw input. By prompting this transformer to answer targeted contextual questions—such as "When does the reader not support?" and "When is there an invocation of mark()?"—the system translates the ambiguous sentence into a structured, explicit rule. An example structured response is as follows:

> *Context:*

Mark Supported: Invocation of `Reader.markSupported()` that evaluates to false. Mark Used: Invocation of `Reader.mark(int readAheadLimit)`.

Structured Rule:

IF Mark Unsupported is true for a given `Reader` instance, THEN Mark Used MUST NOT occur on that instance.

`java.io.Reader.markSupported()`: "Tells whether this stream supports the `mark()` operation. The default implementation always returns false."

`java.io.Reader.mark(int)`: "Not all character-input streams support the `mark()` operation. [...] Throws: IOException - If the stream does not support `mark()`, or if some other I/O error occurs."

For code clarification, we perform a similar strategy. Feeding raw, complex code signatures, such as:

```
call(void Iterator+.remove()) & target(i)
```

directly into the final generator creates a heavy cognitive load. As the context window fills with syntax rather than semantics, the reasoning performance of the LLM demonstrably decreases.

To mitigate this, the Event Interpreter transformer is utilized to translate the raw code signatures into plain English descriptions. The complex Java method call is abstracted and simplified into a straightforward sentence:

Remove: The iterator i has remove() called.

By deploying these two independent clarification transformers, the pipeline effectively normalizes the data. It converts both vague human documentation and rigid machine code into a unified, highly structured natural language format. This ensures that the final ERE Generator (Transformer 3) receives optimal, easily digestible context, significantly reducing the likelihood of logic hallucinations.

4.5 Final Program Structure

Here is the diagram of the final architecture:

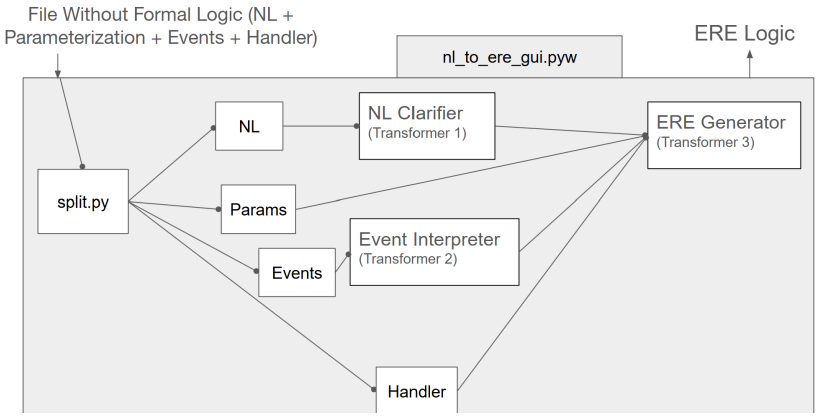

Figure 2: Flowchart showing how NL, Parameters, Events, and the Handler are passed into the tool to generate the final ERE

5 Results and Discussion

5.1 Test Data

To rigorously evaluate the performance of our automated translation pipeline, establishing a reliable baseline was essential. Because there is a severe lack of large-scale datasets specifically mapping natural language to Extended Regular Expressions (ERE) for Runtime Verification, we utilized a manually curated dataset developed by Legunsun [3].

This dataset is comprised of 137 annotated properties sourced directly from official Java API documentation (Javadocs). Each entry in the database pairs a natural language description of a software constraint—such as "Warns if an obsolete class Dictionary is used"—with its corresponding, expert-written formal MOP specification. These complete .mop files contain the full Parameterization, Events, Logic, and Handlers required for monitor execution.

The dataset was leveraged by splitting the files: the natural language descriptions, along with their defined MOP parameters and events, were fed into our multi-transformer pipeline as raw input. Meanwhile, the expert-written EREs from the dataset were strictly isolated and utilized as the "expected" ground truths. By routing both the LLM-generated ERE and the dataset's ground-truth ERE into our Semantic Comparison Engine, we were able to mathematically verify the pipeline's generation accuracy against a robust, real-world set of software specifications.

5.2 Testing Structure

Here is a diagram of the testing architecture:

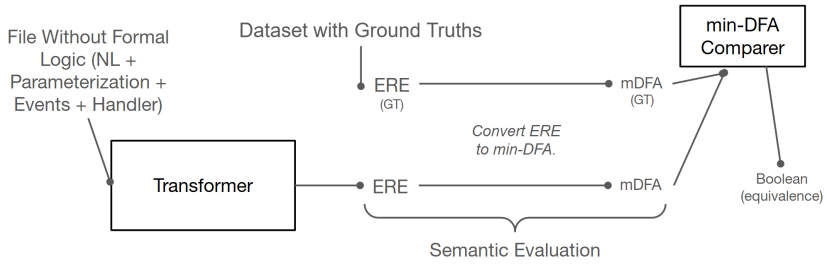

Figure 3: Flowchart showing how generated ERE is tested for semantic similarity

5.3 Testing and Results

Table 2: Performance of Evaluated Models on ERE Generation

Model Configuration	**Semantic Accuracy**	**Syntact**
GPT 5.5 w/ Iterative Prompt	0.42	
GPT 5.5 w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.61	
GPT-OSS w/ Iterative Prompt	0.22	
GPT-OSS w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.28	
Fine-tuned GPT-OSS w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.35	

These are the results to the testing of multiple strategies.

6 Conclusion

6.1 Summary of Findings

This research demonstrates that automating the translation of natural language to Extended Regular Expressions (ERE) for Runtime Verification is a viable, albeit highly complex, endeavor. The data reveals two critical insights regarding model performance: the necessity of high-level reasoning capabilities and the profound impact of modular context clarification.

The OpenAI GPT 5.5 model significantly outperformed the open-source, locally fine-tuned model (GPT-OSS) across all metrics. More importantly, the results validated the hypothesis that bridging the semantic gap between human phrasing and code syntax is critical for LLM performance. When GPT 5.5 was run using only iterative prompting, it achieved a 42% semantic accuracy. However, when the input was preprocessed through our dual-transformer

pipeline—utilizing the NL Clarifier and the Event-to-NL Interpreter—the semantic accuracy surged to 61%. By normalizing vague Javadocs and raw code signatures into explicit, structured English beforehand, we drastically reduced the cognitive load on the final ERE Generator, allowing it to focus entirely on logical mapping rather than context deciphering.

Finally, the results across all models universally proved the absolute necessity of the minimal deterministic finite automata (min-DFA) Semantic Comparison Engine. In every test configuration, semantic accuracy was notably higher than syntactic accuracy (e.g., GPT 5.5 achieved 61% semantic vs. 44% syntactic accuracy). Had this project relied solely on traditional string-matching (syntactic similarity) to verify the logic, the models' success rates would have been artificially deflated, incorrectly marking dozens of functionally perfect EREs as failures simply because they utilized structurally different, yet mathematically equivalent, pathways.

6.2 Project Impact

The fundamental motivation behind this research was to address the steep barrier to entry that prevents the widespread adoption of robust software monitoring. Writing formal logic is a complex, error-prone task that requires highly specialized mathematical knowledge. By successfully automating the translation of intuitive natural language into rigorous Extended Regular Expressions (ERE), this project directly addresses that bottleneck.

The development of this dual-transformer clarification pipeline, paired with the automated minimal deterministic finite automata (min-DFA) semantic comparison engine, proves that Large Language Models can be effectively leveraged to write software specifications. Ultimately, this automation significantly reduces the barrier-to-entry for utilizing Monitoring-Oriented Programming (MOP) and its associated runtime-verification tools [6, 7]. By removing the need for developers to manually write complex state-machine logic, this project makes efficient, real-time, and rigorous software debugging much more accessible to the average software engineer, paving the way for safer and more reliable codebases.

6.3 Future Work

Future iterations of this project should conduct a broader survey of emerging Large Language Models. Testing more advanced reasoning models (such as OpenAI's specialized reasoning series) or open-source models explicitly fine-tuned on larger, newly curated datasets of formal logic may yield even higher semantic accuracy.

Currently, the pipeline is strictly scoped to generating Extended Regular Expressions (ERE) for MOP. Future work will attempt to adapt the generator and the semantic comparison engine to support other complex forms of formal logic, such as Linear Temporal Logic (LTL), allowing the tool to integrate with a wider variety of runtime verification frameworks.

The current system was evaluated against annotated Java API (Javadoc) documentation. To truly validate the tool's efficacy, the next step is to deploy it within a real-world industry setting. Testing the pipeline against the highly complex, proprietary natural language requirements of an active enterprise codebase will provide better insight into its practical limitations.

References